

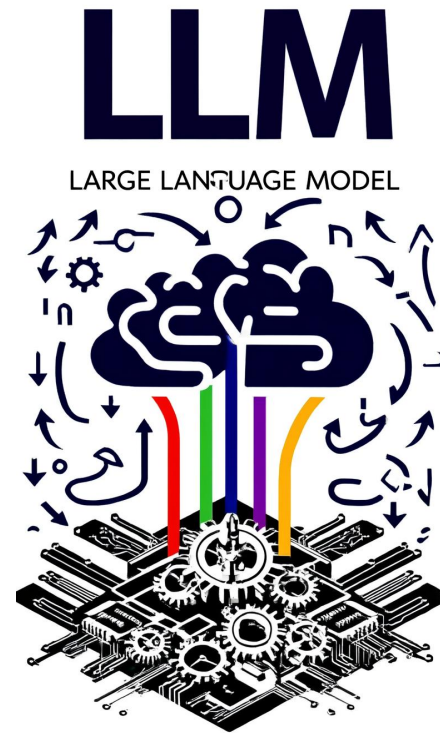
Optimizations of LLM Serving: From an Algorithmic Perspective

Junhao Hu
Peking University

LLMs Reshape Many Applications



可灵AI 2.0



All AI-generated figures in this talk are generated by Kling AI

Large-Scale Deployment of LLMs is Challenging

- Low latency
- High throughput
- Better hardware utilization
-



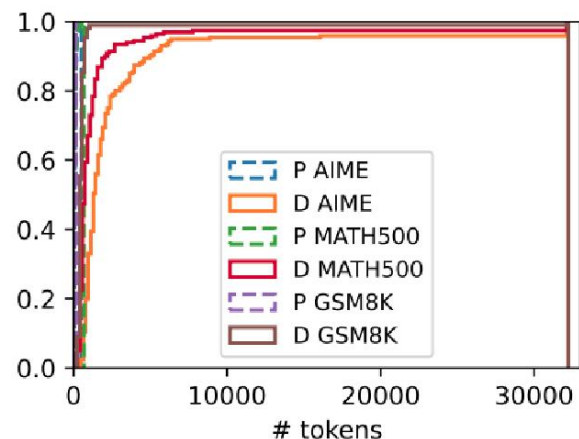
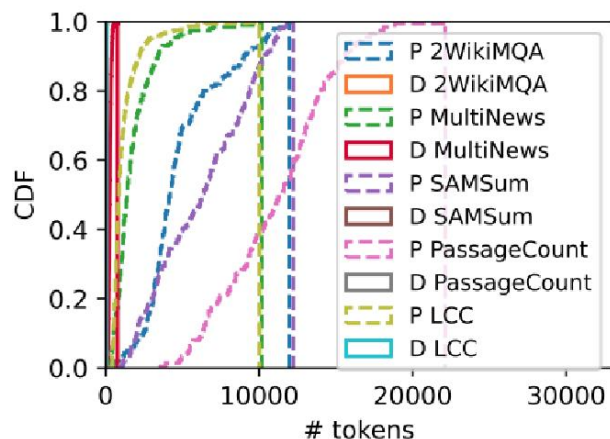
Some Reasons

- Input

- Long few-shot examples, e.g., demonstration of tool usage
- Long documents in Retrieval-Augmented-Generation
- Repository-level information
-

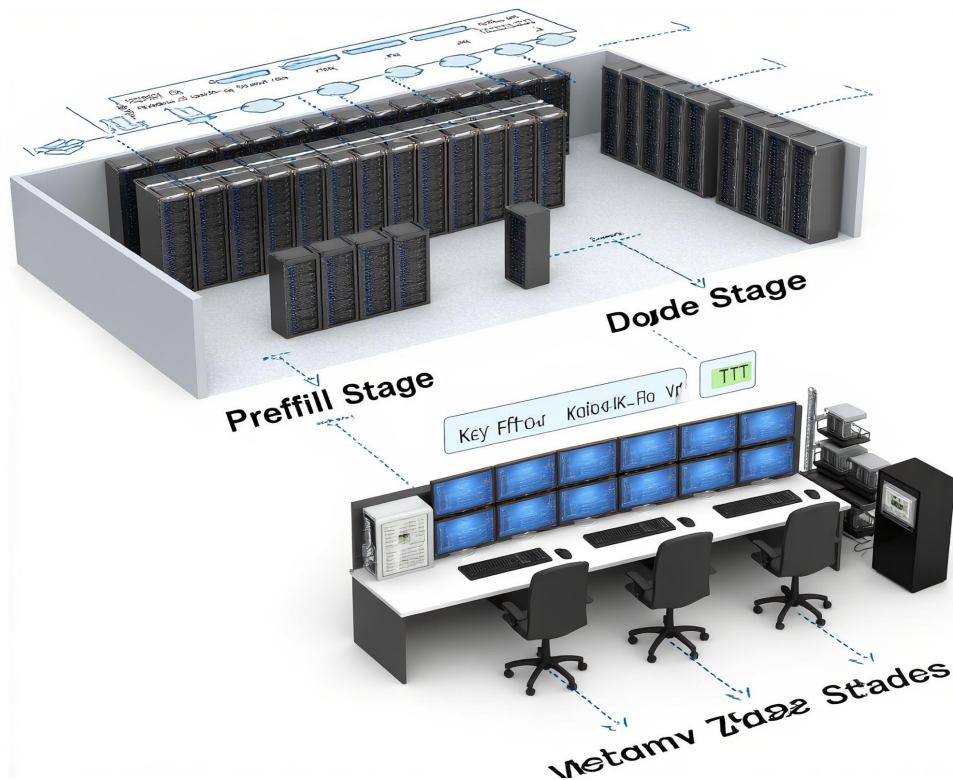
- Output

- Long serial reasoning
- Multiple parallel reasoning
-



Input/Output Challenges vs. Prefill/Decode Stages

- **Prefill**: the model processes prompt tokens all at once, computing their Key (K) and Value (V) cache, and generates the first output token to initiate the decode stage.
- **Decode**: the model iteratively processes each newly generated token, computing its KV cache, and generates the next token.



Content

- EPIC: Efficient Position-Independent Caching for Serving Large Language Models
- RaaS: Reasoning-Aware Attention Sparsity for Efficient LLM Reasoning

Next, we discuss two papers dealing with input/prefill and output/decode optimizations, respectively.

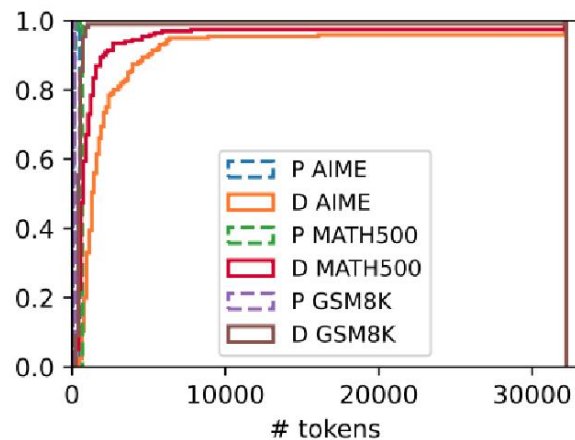
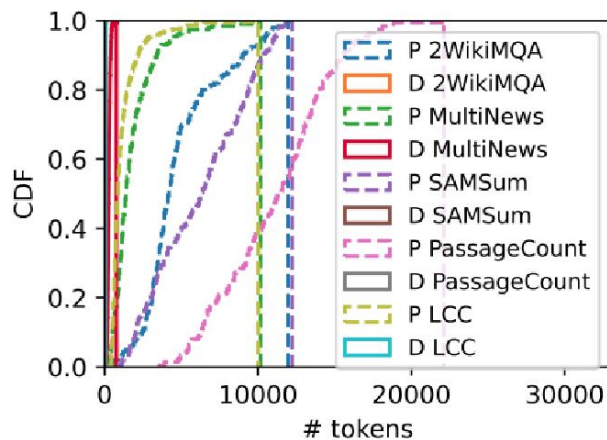
EPIC: Efficient Position-Independent Caching for Serving Large Language Models

Junhao Hu¹, Wenrui Huang², Weidong Wang², Haoyi Wang¹, Tiancheng Hu¹, Qin Zhang³
Hao Feng³, Xusheng Chen³, Yizhou Shan³, Tao Xie¹

Peking University¹, Nanjing University², Huawei Cloud³

Complicated Inputs/Prompts

- Input
 - Long few-shot examples, e.g., demonstration of tool usage
 - Long documents in Retrieval-Augmented-Generation
 - Repository-level information
 -



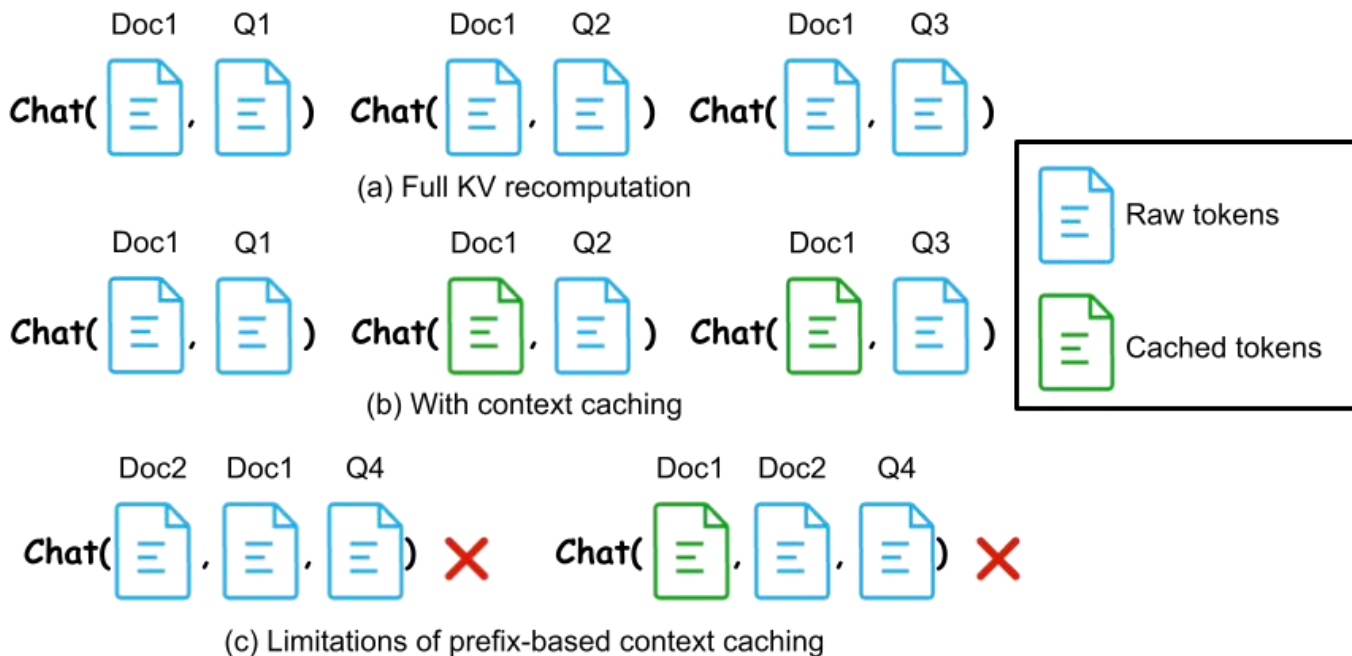
Context Caching Comes to Rescue

In LLM serving, immutable chunks (like system messages, few-shot examples, and documents) are frequently repeated across requests.

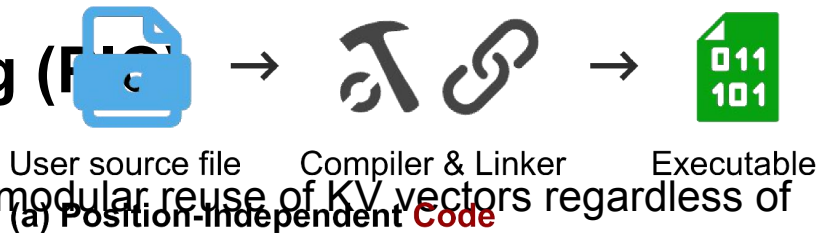
Traditional context caching (Prefix-Based Context Caching) reuses KV vector, saving computation time in the prefill stage. → Time-to-First-Token (TTFT)↓

Limitations of Prefix Matching

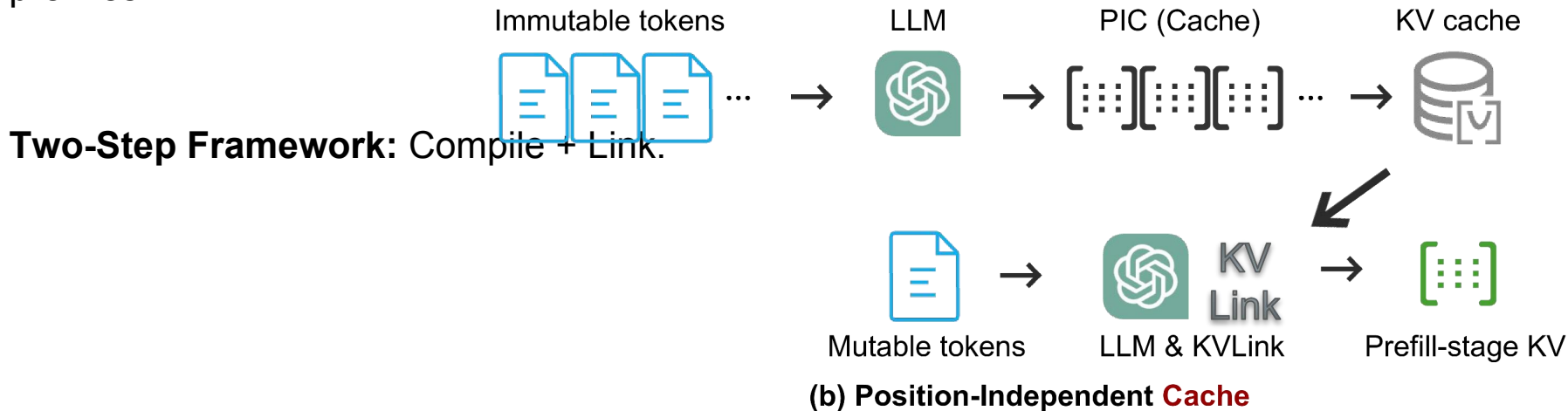
Existing context caching methods fail to efficiently reuse immutable chunks when preceded by varying prefixes.



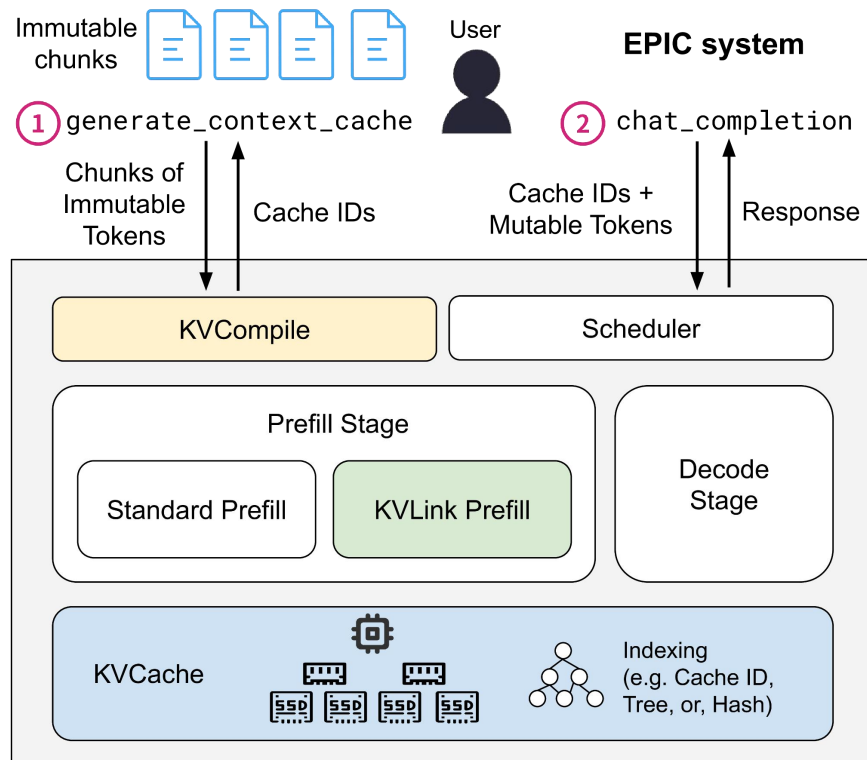
Position-Independent Caching (PIC)



Position-Independent Caching (PIC) enables modular reuse of KV vectors regardless of prefixes.

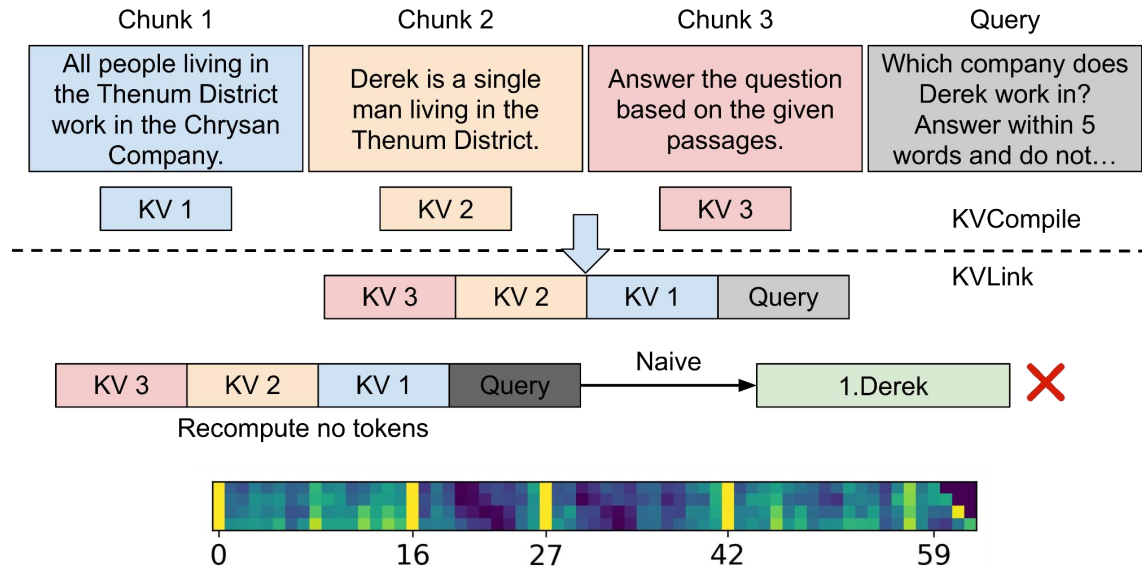


The EPIC Serving System Following the Compile+Link Design



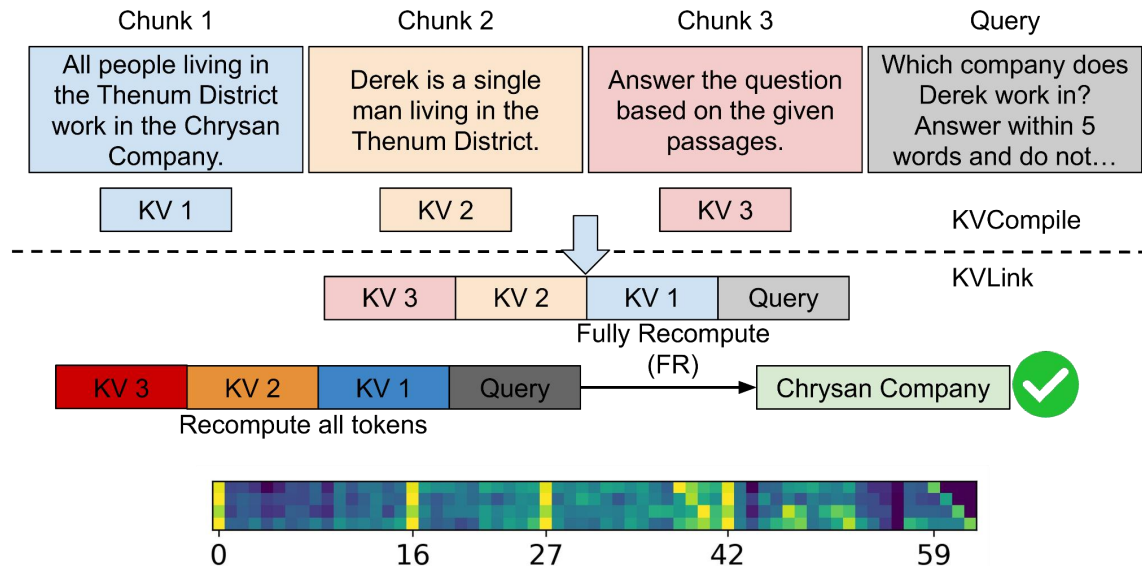
Approaches for PIC — *Naive*

Naive: $O(1)$ link time; low accuracy. Most attention scores concentrate on each chunk's initial tokens, exhibiting the “attention sink” phenomenon.



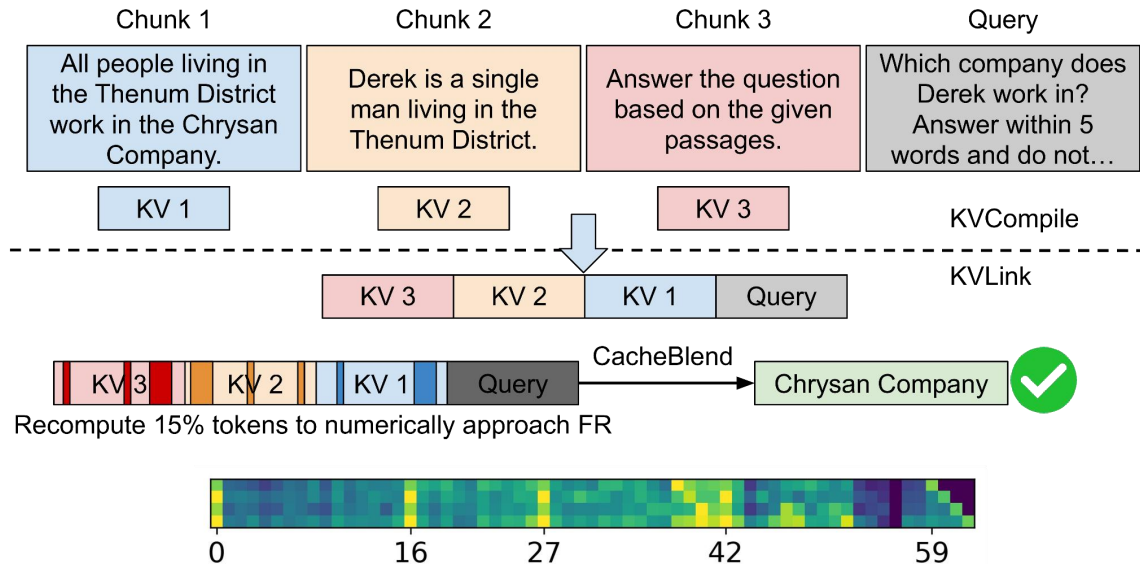
Approaches for PIC — *Fully Recompute*

Fully Recompute (FR): $O(N^2)$ link time; full accuracy. Each chunk's initial tokens release part of their attention to more relevant positions.



Approaches for PIC — *CacheBlend*

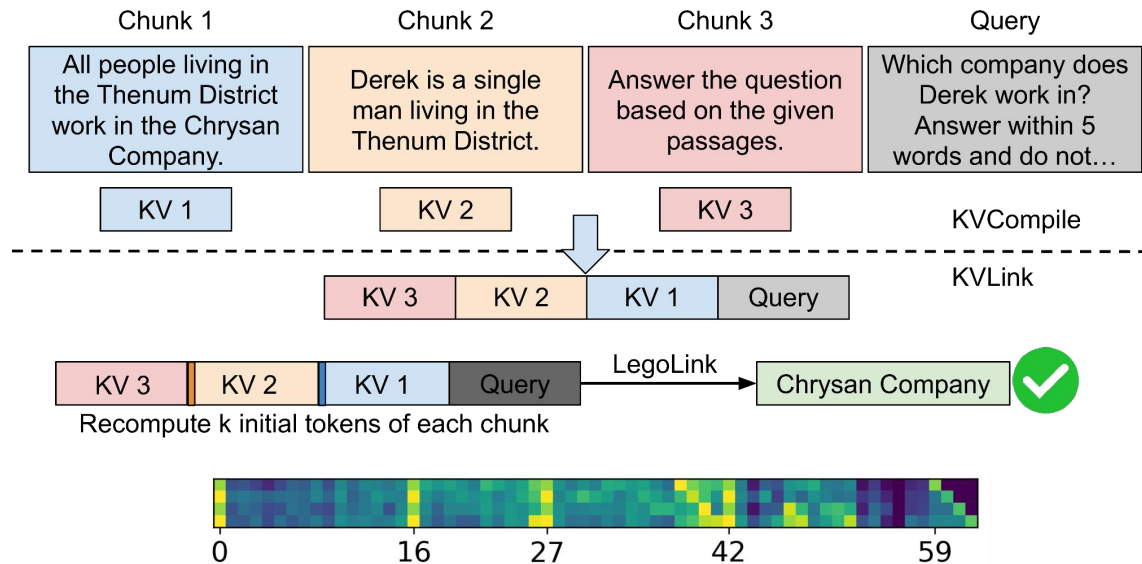
CacheBlend: $O(15\%N^2)$ link time; ~full accuracy. *CacheBlend* approximates FR's attention map by selectively recomputing only 15% of tokens with the largest deviation from the FR. These selected tokens often include initial tokens of each chunk



Our Approach – *LegoLink*

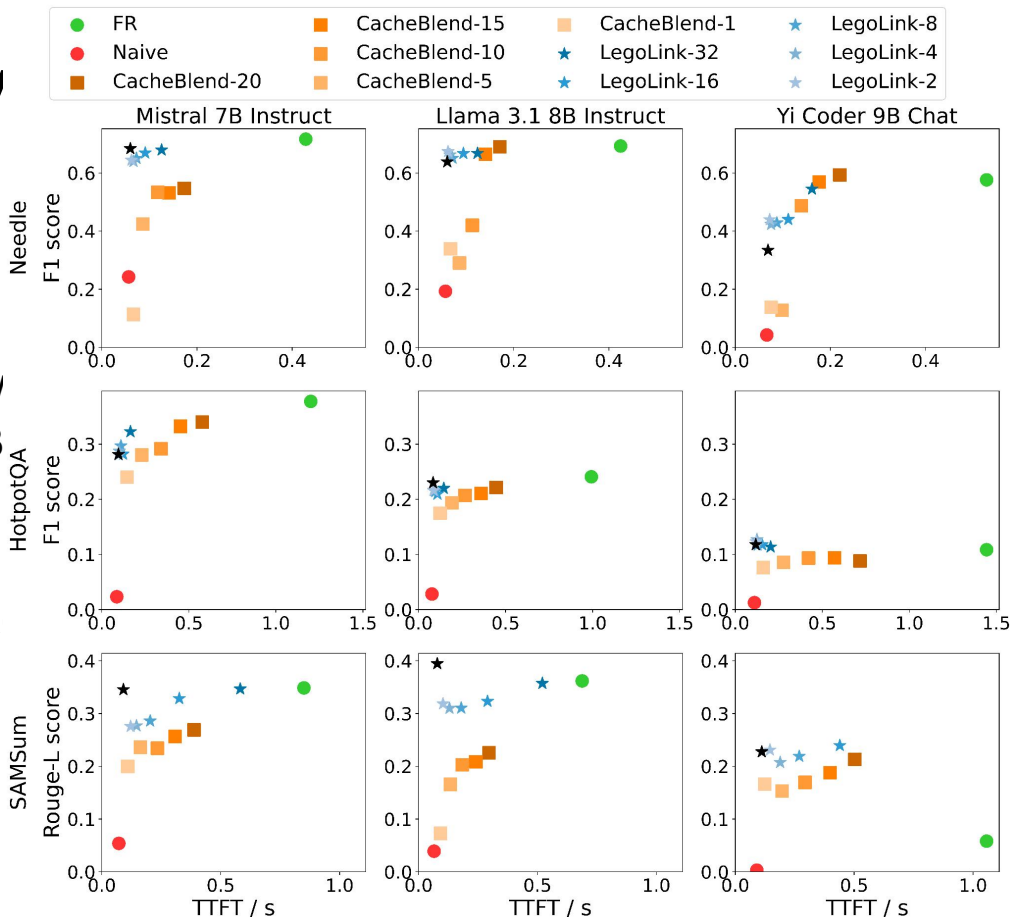
LegoLink: $O(kN)$ link time; ~full accuracy. *LegoLink* allows initial tokens of latter chunks to recognize their non-initial positions and crippling their attention-sink ability

Next: More details for *LegoLink*



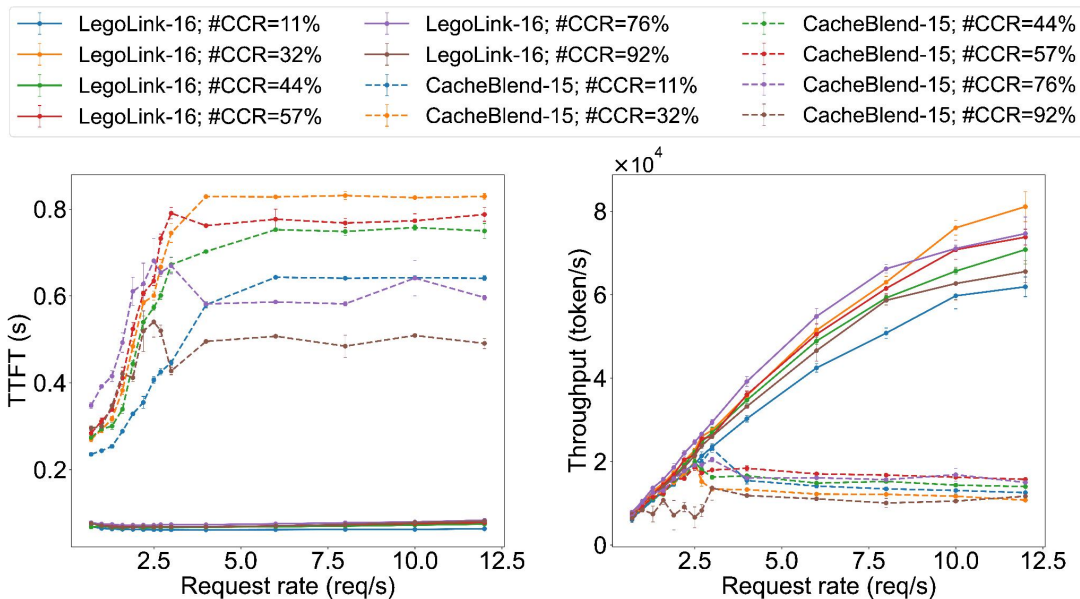
Evaluation — Accuracy v

- *LegoLink* variants establish a new Pareto frontier, outperforming *CacheBlend* in most cases
- *LegoLink-2* limits accuracy drops w 0-7% and reduces TTFT by up to 3 compared to *CacheBlend-15*
- Increasing recomputed tokens in *LegoLink* yields diminishing accuracy gains



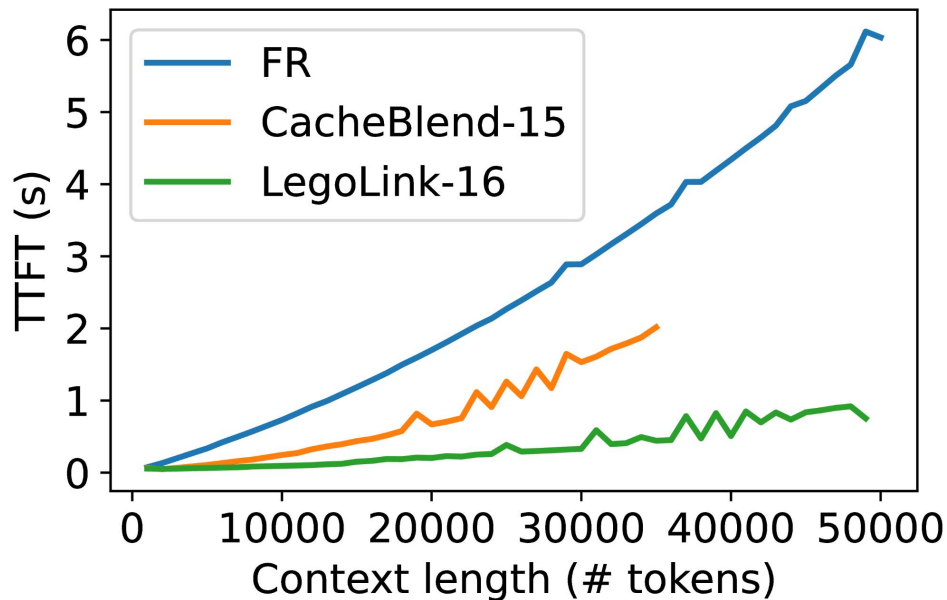
Evaluation — Latency and Throughput

- EPIC achieves up to 8× reduction in TTFT and 7× increase in throughput compared to existing systems.
- Under asynchronous workloads, EPIC maintains stable performance as Context Cache Ratio (CCR) increases.



Evaluation — Latency Under Long Context

- EPIC supports longer context lengths with smaller latency, without out-of-memory errors



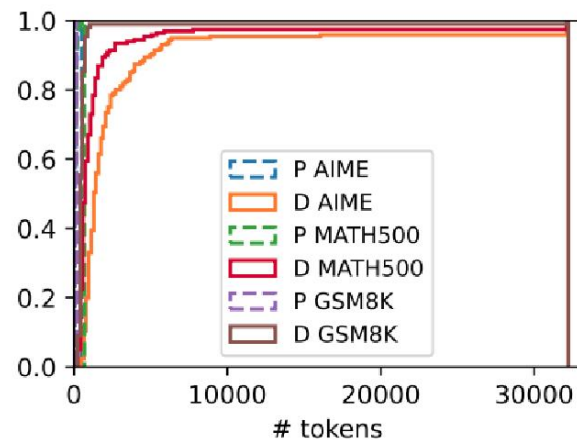
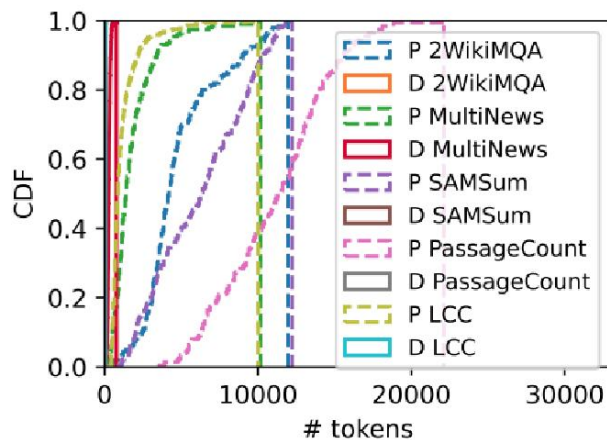
RaaS: Reasoning-Aware Attention Sparsity for Efficient LLM Reasoning

Junhao Hu¹, Wenrui Huang², Weidong Wang², Zhenwen Li¹, Tiancheng Hu¹,
 Zhixia Liu³, Xusheng Chen³, Tao Xie¹, Yizhou Shan³

Peking University¹, Nanjing University², Huawei Cloud³

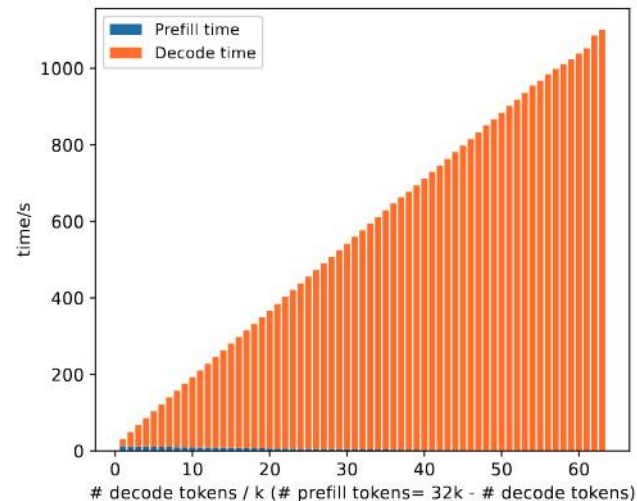
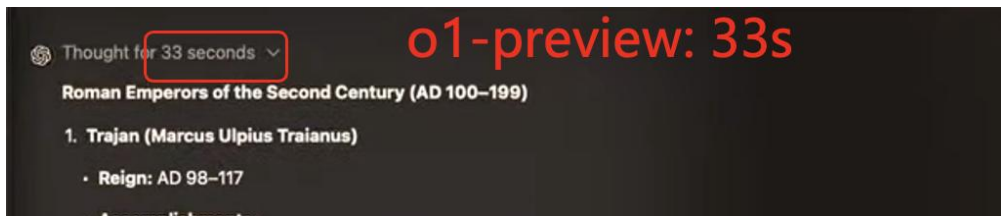
Complicated Outputs

- Output
 - Long serial reasoning
 - Multiple parallel reasoning
 -



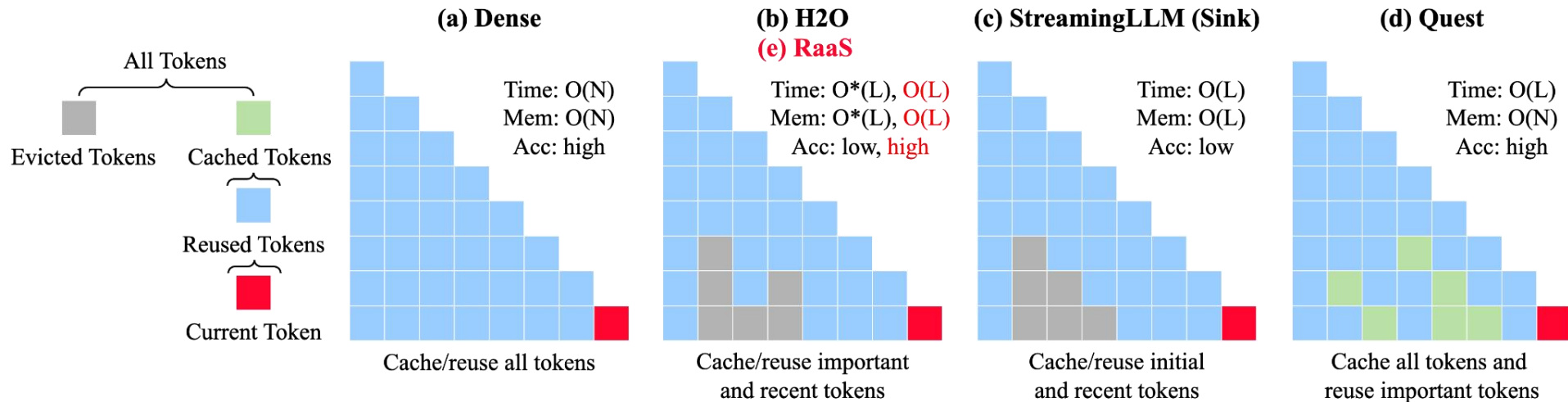
Time and Memory Complexities of LLM Inference

- Generate **one token**: $O(N)$ time and $O(N)$ memory complexities. N denotes the current sequence length (i.e., input plus output tokens).
- Completing an **entire request**: a total time complexity of $O(N^2)$.
- Example: the Llama 3.1 8B model supports sequences up to 128,000 tokens, leading to Job Completion Times (JCT) of several thousand seconds and memory usage up to 16GB per request.



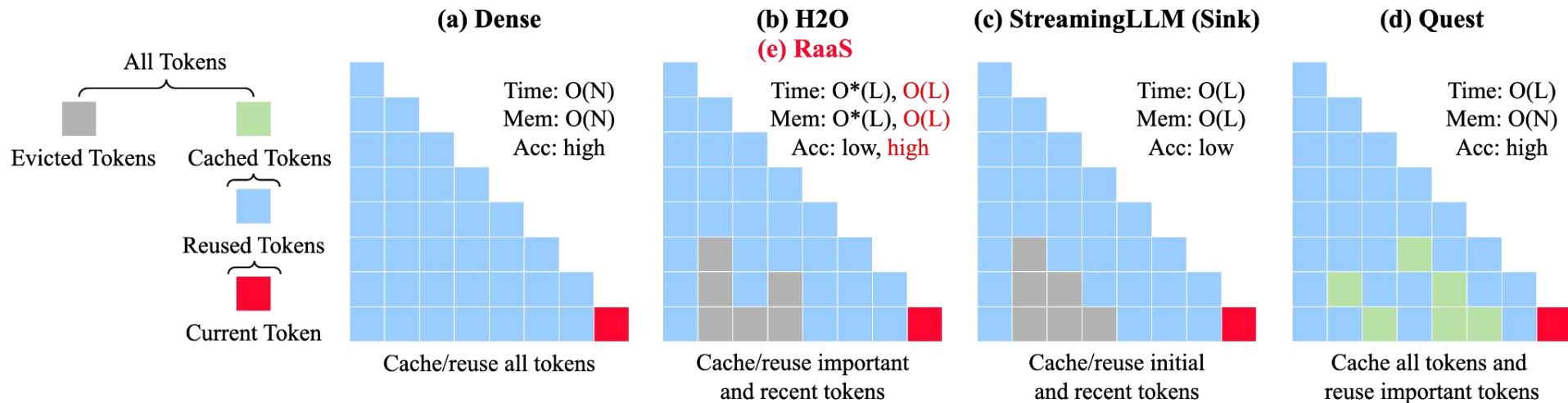
Existing Sparsity-Based Algorithms

To reduce time and memory complexities of long-decode inference, one line of research uses sparsity-based algorithms that retain the KV vectors of only the most critical tokens. But these algorithms struggle with the “**impossible trinity**” of accuracy, time, and memory. (L indicates the cache size or cache budget.)



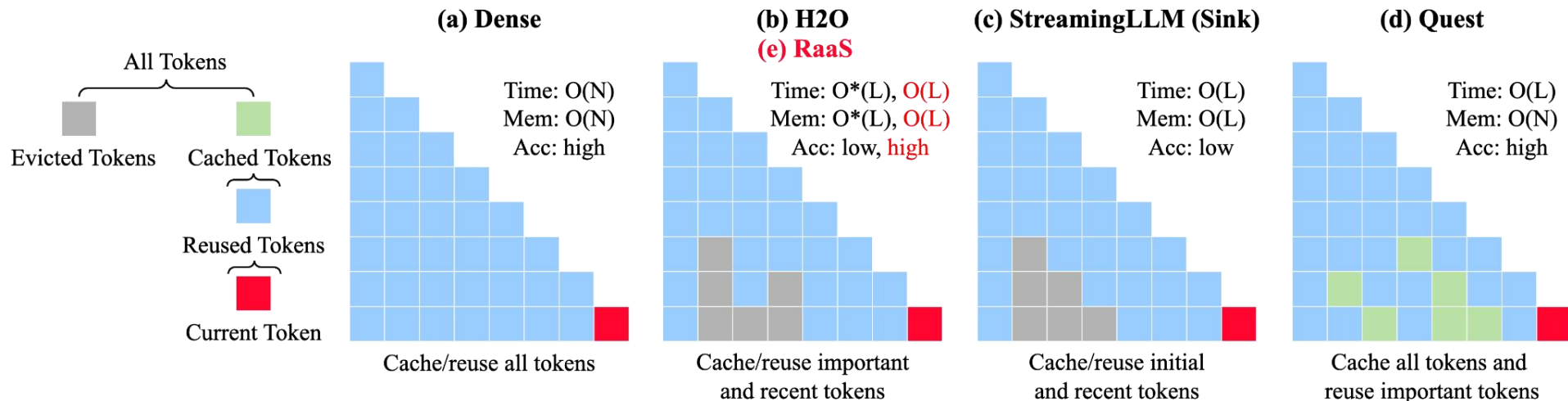
Existing Sparsity-Based Algorithms

(a) **Dense**, the standard attention algorithm, caches and reuses KV vectors of all tokens.



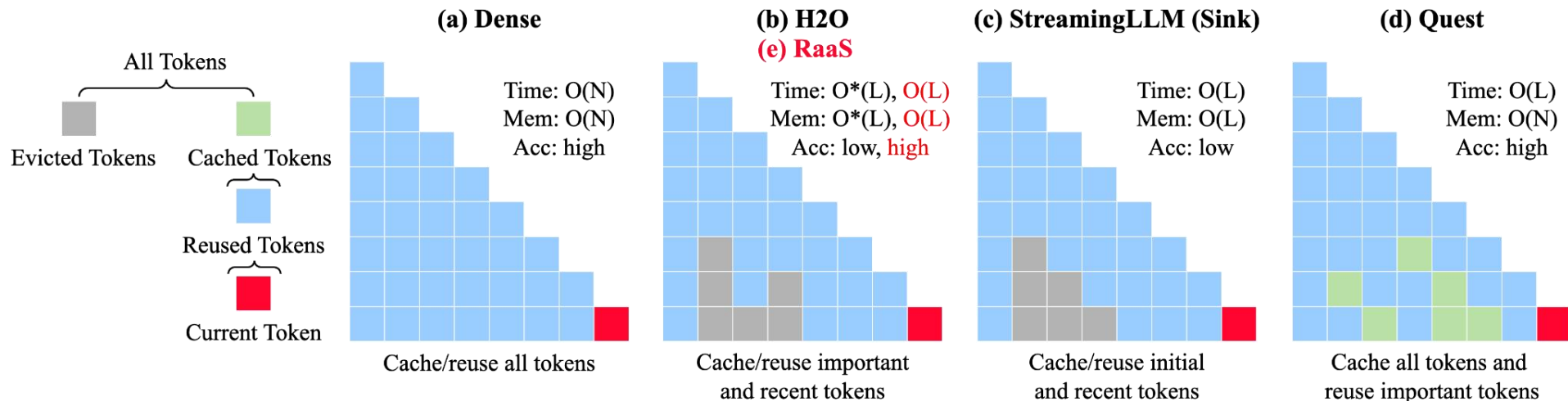
Existing Sparsity-Based Algorithms

(b) H2O, the pioneering sparsity-based algorithm, caches and reuses KV vectors of recent tokens and important non-recent tokens. When the cache is full, it evicts non-recent tokens with the lowest accumulated attention scores.



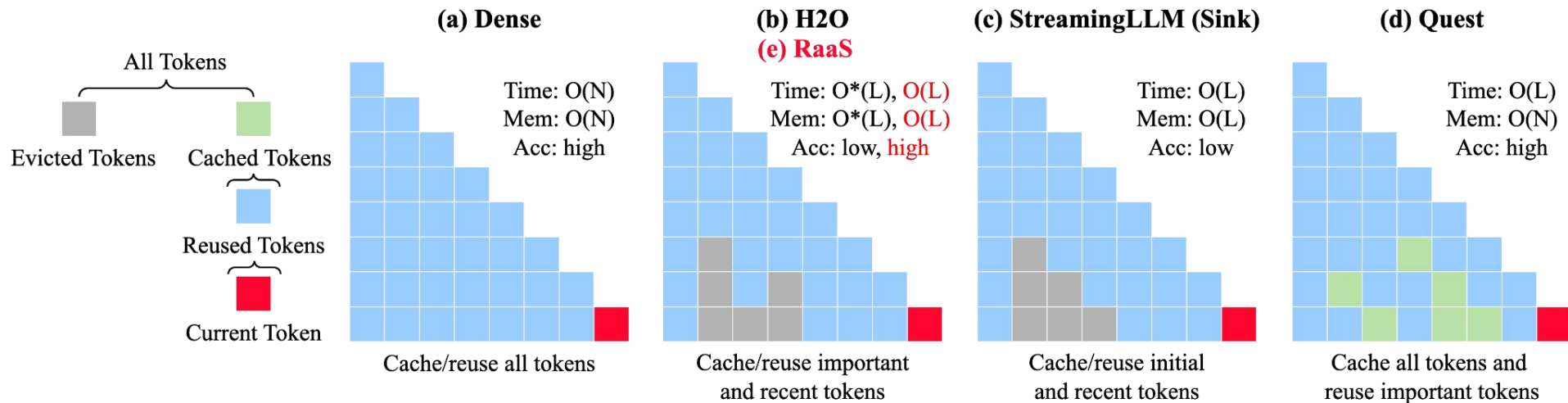
Existing Sparsity-Based Algorithms

(c) **StreamingLLM**, or the Sink algorithm, statically decides to cache and reuse KV vectors of only the initial and recent tokens, without dynamically selecting important ones based on attention scores as H2O.



Existing Sparsity-Based Algorithms

(d) **Quest**, the state-of-the-art algorithm, caches KV vectors of all tokens but reuses only the ones with the top-k attention scores.



Opportunities to Break the Impossible Trinity

A new attention pattern.

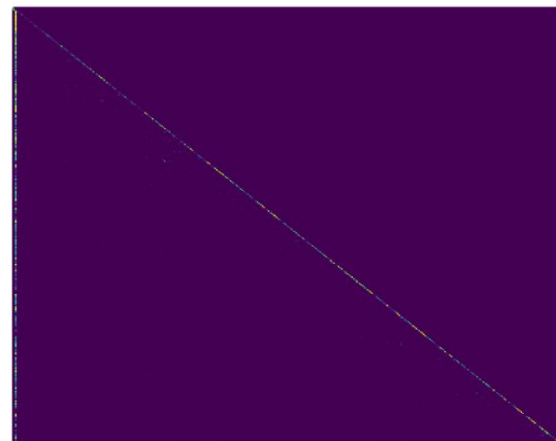
- **Milestone tokens** (analogous to lemmas in mathematical proofs) emerge, are utilized, and then become unimportant.
- **Phoenix tokens** receive low attention scores for a period long enough to be evicted from cache but later regain importance.



(a) Milestone pattern (24.2%)



(b) Phoenix pattern (1.5%)



(c) StreamingLLM pattern (74.3%)

Examples

To convert the point $((0,3))$ from rectangular coordinates to polar coordinates, we need to find the values of (r) and (θ) . The formulas for converting from rectangular coordinates $((x,y))$ to polar coordinates $((r,\theta))$ are:

$$[r = \sqrt{x^2 + y^2}] \text{ ①}$$

$$[\theta = \tan^{-1}\left(\frac{y}{x}\right)] \text{ ②}$$

Given the point $((0,3))$, we have $(x = 0)$ and $(y = 3)$. Let's calculate (r) first: ③

$$[r = \sqrt{0^2 + 3^2} = \sqrt{9} = 3]$$

Next, we need to find (θ) . The formula $(\theta = \tan^{-1}\left(\frac{y}{x}\right))$ is not directly useful here because it involves division by zero, which is undefined. Instead, we need to consider the position of the point $((0,3))$ in the coordinate plane. The point $((0,3))$ lies on the positive (y) -axis. Therefore, the angle (θ) is $(\frac{\pi}{2})$.

So, the polar coordinates of the point $((0,3))$ are: ④

$$[(r, \theta) = \left(3, \frac{\pi}{2}\right)] \text{ ⑤}$$

Thus, the final answer is: ⑥

$$[\boxed{\left(3, \frac{\pi}{2}\right)}]$$

Prompt: If $f(x) = \frac{3x-2}{x-2}$, what is the value of $f(-2) + f(-1) + f(0)$? Express your answer as a common fraction.

Decode:

To find the value of $(f(-2) + f(-1) + f(0))$ for the function $(f(x) = \frac{3x-2}{x-2})$, we need to evaluate $(f(x))$ at $(x = -2)$, $(x = -1)$, and $(x = 0)$ separately and then sum the results. ①

First, let's evaluate $(f(-2))$:

$$[f(-2) = \frac{3(-2) - 2}{-2 - 2} = \frac{-6 - 2}{-4} = \frac{-8}{-4} = 2] \text{ ②}$$

Next, let's evaluate $(f(-1))$:

$$[f(-1) = \frac{3(-1) - 2}{-1 - 2} = \frac{-3 - 2}{-3} = \frac{-5}{-3} = \frac{5}{3}] \text{ ③}$$

Then, let's evaluate $(f(0))$:

$$[f(0) = \frac{3(0) - 2}{0 - 2} = \frac{-2}{-2} = 1] \text{ ④}$$

Now, we sum the values of $(f(-2))$, $(f(-1))$, and $(f(0))$:

$$[f(-2) + f(-1) + f(0) = 2 + \frac{5}{3} + 1] \text{ ⑤}$$

To add these numbers, we need a common denominator. The common denominator for 2, $(\frac{5}{3})$, and 1 is 3. We convert 2 and 1 to fractions with a denominator of 3:

$$[2 = \frac{6}{3} \quad \text{and} \quad 1 = \frac{3}{3}]$$

Now we can add the fractions: ⑥

$$[\frac{6}{3} + \frac{5}{3} + \frac{3}{3} = \frac{6 + 5 + 3}{3} = \frac{14}{3}] \text{ ⑦}$$

Therefore, the value of $(f(-2) + f(-1) + f(0))$ is $(\frac{14}{3})$

Possible Explanations

A new attention pattern.

- **Milestone tokens** are analogous to lemmas in mathematical proofs or subconclusions in thinking steps. Once an LLM generates milestone tokens, subsequent tokens primarily attend to the milestone tokens rather than the preceding tokens arriving at the milestone tokens.
- **Phoenix tokens** rarely appear in decode tokens because decode tokens primarily consist of milestone tokens, but frequently occur in prefill tokens because LLMs typically refer back to the front user queries in the final conclusions of reasoning.

Theoretical explanations are still missing

The RaaS Algorithm

- We identify milestone tokens and retain their KV vectors using a Least-Recently-Used (LRU) caching strategy.
 - During each decoding step, tokens that receive attention scores above the median are considered **used** and are assigned the latest timestamp.
 - Milestone tokens typically continue to receive the latest timestamps until they become permanently irrelevant. When the KV cache is full, RaaS evicts KV vectors of the tokens with the oldest timestamp.

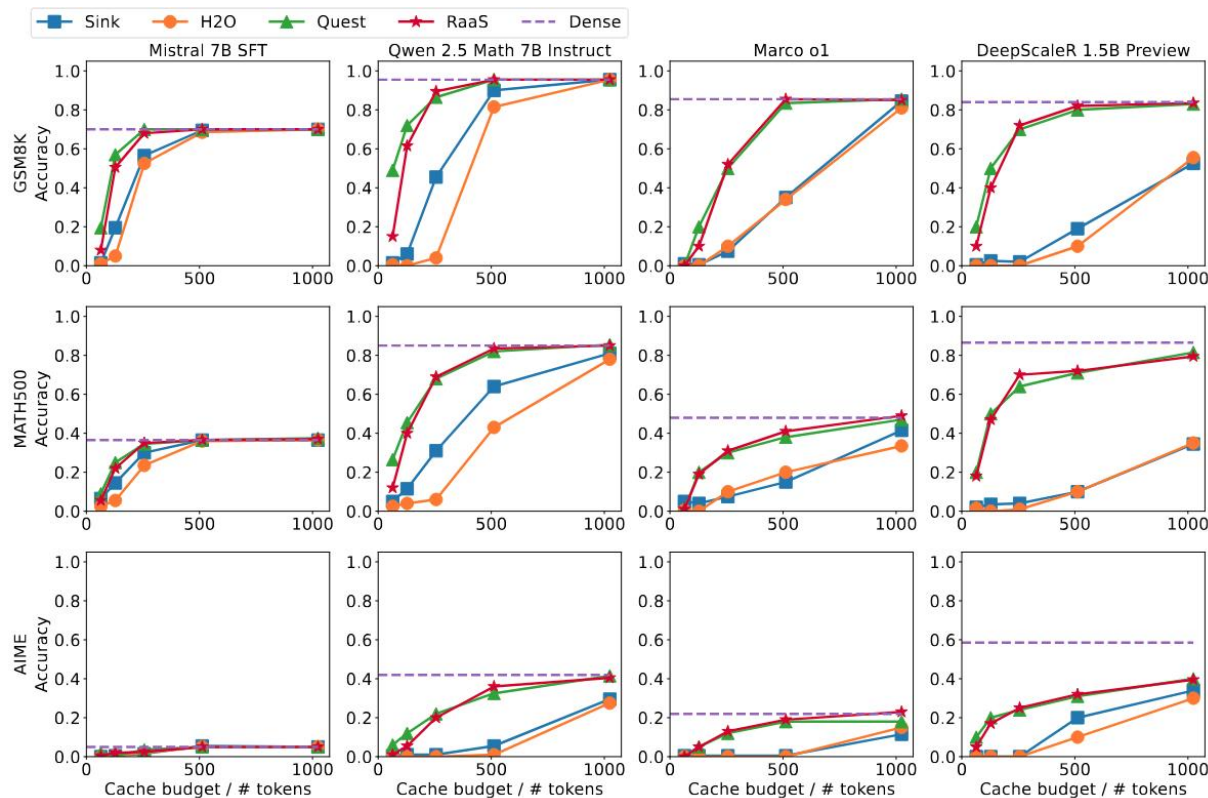
- We retain the KV vectors of all prefill tokens without eviction.

- Page-based

```
Function kvm.store_cache(K, V, score, stage)
if stage == “prefill”
    Store K, V and mark them as non-evictable
else if stage == “decode”
    Store K_new and V_new.
    New timestamps to KVs with top-r score
    if kvm.is_full()
        Evict KVs with the oldest timestamps
End Function
```

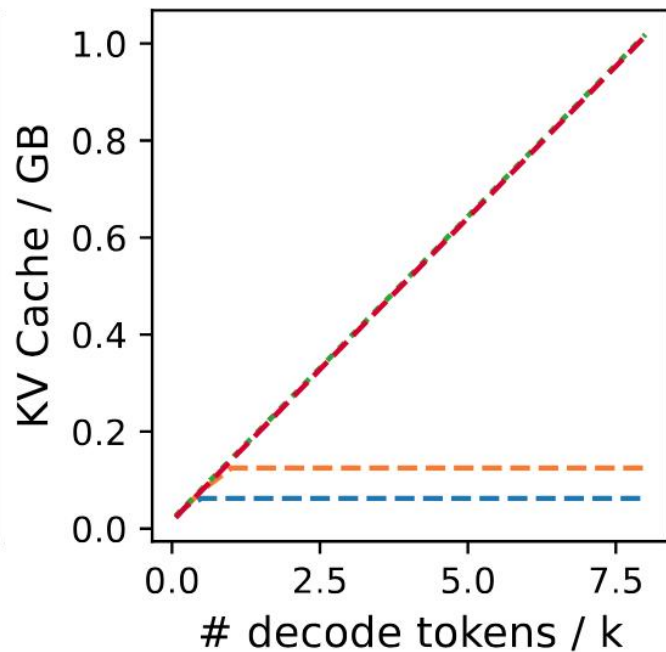
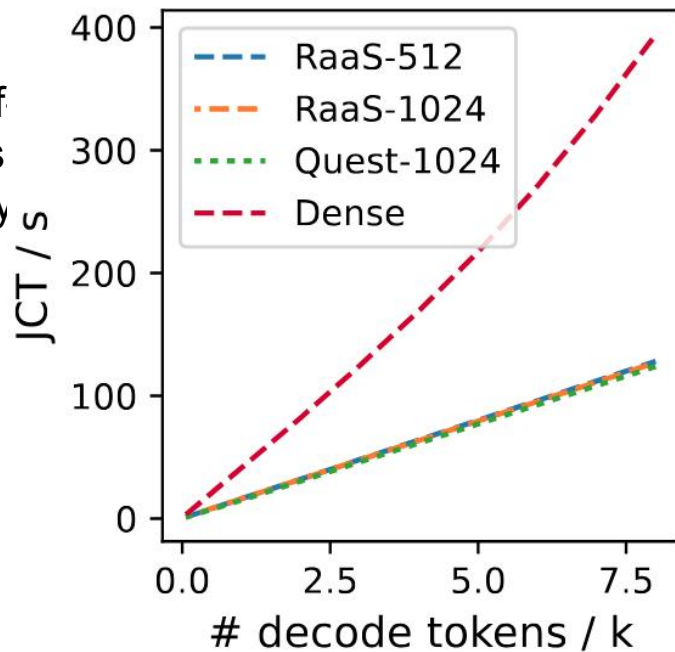

Accuracy vs. Cache Budget

RaaS achieves the same level of accuracy as the state-of-the-art Quest with 512-token cache budget.



Latency and Throughput

But RaaS has only $O(L)$ memory complexity compared to the state-of-the-art Quest, which has $O(N)$ memory complexity



Key Takeaway

- Input/Prefill: Position-Independent Cache (PIC) increases cache hit rate and decrease TTFT in the prefill stage.
- Output/Decode: Milestone tokens, which are analogous to lemmas in math proofs or subconclusions in thinking steps, help reduce cache size and computation costs in the decode stage.



Optimizations of LLM Serving: From an Algorithmic Perspective



Junhao Hu

junhaohu@stu.pku.edu.cn



Wenrui Huang

wr-huang@outlook.com



Weidong Wang

kenazcharisma@gmail.com



Tao Xie

taoxie@pku.edu.cn



Yizhou Shan

shanyizhou@huawei.com

Thank You!

Q&A